

SLIIT Faculty of Computing

IT3012: Intelligent Agents

Practical 03: Goal-Based Search Agent

BSc (Hons) in Information Technology Specialized in AI

Year 3 - Semester 1 - 2026 Semester 2

Implementation Summary

The environment exposes its complete state through `get_percept()`:

- `grid_size`: (width, height)
- `walls`: the set of blocked cells
- `all_food`: every food coordinate

SearchAgent implements graph-search versions of BFS, DFS, and UCS. The selected algorithm is controlled by `active_algo`, which can be set to "BFS", "DFS", or "UCS". The agent creates an offline movement plan, executes one action per percept cycle, and performs suck when it reaches the food.

Theoretical Evaluation

1. State Space vs Search Tree

The state space is the complete set of unique environment states that can be reached, such as valid grid coordinates and their possible configurations.

The search tree is the structure generated by an algorithm while exploring that state space from one initial state. A single state can appear more than once in a search tree through different paths, while it represents only one unique state in the state space.

2. Purpose of the reached Set

The reached set prevents the algorithm from expanding the same state more than once. It converts tree search into graph search, prevents repeated work, and stops cycles from causing infinite exploration. Without it, DFS could keep following loops between neighboring cells and might never reach the food.

3. BFS Compared with DFS

BFS expands states level by level. Because every legal grid move has the same cost, the first path BFS finds has the fewest moves and is therefore optimal.

DFS follows one branch as deeply as possible before backtracking. Its first solution depends on neighbor ordering, so it can take a winding path even when a shorter path exists. DFS is not guaranteed to return an optimal path.

UCS also finds an optimal path here because every move has cost 1. In this uniform-cost grid, its result has the same path length as BFS.

4. BFS and DFS Memory Bottlenecks

For branching factor b and shallowest goal depth d , BFS can require $O(b^d)$ time and $O(b^d)$ space because it stores the whole frontier at the current depth. On a 1000x1000 grid, this frontier and the reached set can consume a large amount of memory before the goal is found.

DFS uses $O(b^m)$ time in the worst case, where m is the maximum search depth, but its frontier normally requires only $O(bm)$ space. Therefore, DFS has a much smaller memory bottleneck than BFS, although it may spend a long time exploring an unhelpful branch and does not guarantee an optimal solution.

Verification

- BFS, DFS, and UCS each produced a valid path on a reachable test grid.
- BFS and UCS produced paths of equal minimum length.
- The simulation reached food, executed suck, and finished with score 20.
- `agent.py` and `visual_grid_game.py` passed Python syntax compilation.